

NETSOL RAG SYSTEM

Complete Backend Master Prompt

Step 0: Environment Setup · Step 1: Ingestion Pipeline · Step 2: Retrieval Pipeline

25,069	602	13.7M	6,528	3
Chunks	PDFs	Words	Web Pages	ChromaDB Collections

LLM: gemini-2.0-flash (all 7 nodes) Embeddings: gemini-embedding-2-preview (MTEB #1 2025) Vector DB: ChromaDB Orchestration: LangGraph API: FastAPI + SSE Streaming

STEP 0

Environment Setup

Install packages · API keys · Project structure · config.py

0.1 Install All Dependencies

TERMINAL — Install

```
# Run in terminal / Google Colab
pip install google-generativeai langchain-google-genai langchain langgraph
pip install chromadb rank-bm25 sentence-transformers
pip install fastapi uvicorn python-dotenv jsonlines tqdm nest-asyncio
```

0.2 Project Structure

PROJECT STRUCTURE

```
netsol_rag/
├── netsol_scraped_data/
│   ├── rag_chunks.jsonl # 25,069 chunks – MAIN INPUT
│   └── rag_corpus.jsonl # 6,319 page records
├── chroma_db/ # Auto-created by ChromaDB
├── bm25_index.pkl # Auto-created by ingest.py
├── .env # API keys
├── config.py # All settings
├── state.py # LangGraph state schema
├── ingest.py # STEP 1 – run once
├── nodes.py # STEP 2 – all 7 LangGraph nodes
├── graph.py # STEP 2 – graph wiring
└── api.py # STEP 2 – FastAPI server
```

0.3 Environment Variables (.env)

.env FILE

```
# .env – create this file in project root
GOOGLE_API_KEY=your_gemini_api_key_here
# Optional – LangSmith tracing (recommended)
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=your_langsmith_key
LANGCHAIN_PROJECT=netsol-rag
```

0.4 config.py — All Settings in One Place

config.py

```
# config.py
import os
from dotenv import load_dotenv
load_dotenv()

# ■■■ API ■■■
GOOGLE_API_KEY = os.getenv('GOOGLE_API_KEY')

# ■■■ MODELS ■■■
EMBED_MODEL = 'gemini-embedding-2-preview' # MTEB #1 2025
LLM_MODEL = 'gemini-2.0-flash' # Fast + free tier

# ■■■ CHROMADB ■■■
CHROMA_PATH = './chroma_db'
COLLECTION_WEB = 'netsol_web_pages'
COLLECTION_PDF = 'netsol_pdfs'
COLLECTION_META = 'netsol_metadata'

# ■■■ DATA ■■■
CHUNK_JSONL = './netsol_scraped_data/rag_chunks.jsonl'
CORPUS_JSONL = './netsol_scraped_data/rag_corpus.jsonl'
BM25_INDEX = './bm25_index.pkl'

# ■■■ PIPELINE SETTINGS ■■■
BATCH_SIZE = 50 # Gemini embedding API safe batch size
TOP_K_RETRIEVE = 20 # Chunks to retrieve before reranking
TOP_K_RERANK = 5 # Final chunks after reranking
MIN_CHUNK_WORDS = 40 # Skip chunks shorter than this
MAX_RETRIES = 2 # Hallucination guard max retries
```

0.5 state.py — LangGraph State Schema

[illegible]

STEP 1

Ingestion Pipeline

ingest.py — Run ONCE · Builds 3 ChromaDB collections + BM25 index

Run order: python ingest.py → Reads rag_chunks.jsonl → Embeds with gemini-embedding-2-preview (RETRIEVAL_DOCUMENT) → Stores in 3 ChromaDB collections → Builds BM25 keyword index.

Time estimate: 25,069 chunks / 50 batch = ~502 API calls. Free tier: 1,500 req/min → ~20-30 minutes total.

1.1 Gemini Embedding Functions

PART 1 — Imports + Embedding Function

```
# ingest.py — COMPLETE FILE
from google import genai
from google.genai import types
import chromadb, jsonlines, time, pickle
from tqdm import tqdm
from datetime import datetime
from config import *

# Initialize clients
client = genai.Client(api_key=GOOGLE_API_KEY)
chroma = chromadb.PersistentClient(path=CHROMA_PATH)

def embed_documents(texts: list[str]) -> list[list[float]]:
    """
    Embed a batch of document chunks.
    MUST use task_type=RETRIEVAL_DOCUMENT for stored chunks.
    Separate from query embedding — different vector space.
    """
    response = client.models.embed_content(
        model=EMBED_MODEL,
        contents=texts,
        config=types.EmbedContentConfig(
            task_type='RETRIEVAL_DOCUMENT'
        )
    )
    return [e.values for e in response.embeddings]
```

1.2 Chunk Filter + Metadata Builder

PART 2 — Filter + Metadata

```
# Patterns to skip – noise chunks
SKIP_URLS = ['/tag/', '/category/', '/page/', '/feed/',
             '/wp-admin/', '#', 'javascript:', 'mailto:']

def should_skip(chunk: dict) -> bool:
    """Return True if chunk is noise – skip from ChromaDB."""
    # Too short
    if chunk.get('word_count', 0) < MIN_CHUNK_WORDS:
        return True
    # Empty text
    if not chunk.get('text', '').strip() or len(chunk.get('text', '')) < 100:
        return True
    # Noise URLs (nav, pagination, feeds)
    url = chunk.get('source_url', '')
    if any(p in url for p in SKIP_URLS):
        return True
    # No title = probably a utility page
    if not chunk.get('page_title', '').strip():
        return True
    return False

def recency_score(date_str: str) -> float:
    """0.5 (old) to 1.0 (recent). Used to boost fresh content."""
    try:
        d = datetime.fromisoformat(date_str.replace('Z', ''))
        days_old = (datetime.utcnow() - d).days
        return round(max(0.5, 1.0 - (days_old / 365) * 0.5), 3)
    except:
        return 0.7

def build_metadata(chunk: dict, source_type: str) -> dict:
    """Build ChromaDB metadata – only str/int/float/bool allowed."""
    meta = {
        'chunk_id': str(chunk.get('chunk_id', '')),
        'source_type': source_type,
        'source_url': str(chunk.get('source_url', '')[0:300]),
        'page_title': str(chunk.get('page_title', '')[0:200]),
        'parent_url': str(chunk.get('parent_url', '')[0:300]),
        'level': int(chunk.get('level', 0)),
        'scraped_date': str(chunk.get('scraped_date', '')),
        'recency_score': recency_score(chunk.get('scraped_date', '')),
        'word_count': int(chunk.get('word_count', 0)),
    }
    # Extra fields for PDFs
    if source_type == 'pdf':
        meta['pdf_name'] = str(chunk.get('pdf_name', '')[0:200])
        meta['page_number'] = int(chunk.get('page_number', 0))
        meta['section_heading'] = str(chunk.get('section_heading', '')[0:200])
        meta['doc_category'] = str(chunk.get('doc_category', ''))
    # Replace None with empty string
    return {k: (v if v is not None else '') for k, v in meta.items()}
```

1.3 Ingest Web Pages Collection

PART 3 — Web Pages Collection

```
def ingest_web_pages():
    col = chroma.get_or_create_collection(
        name=COLLECTION_WEB,
        metadata={'hnsw:space': 'cosine'} # cosine similarity
    )
    print(f'Existing: {col.count()} web chunks')
    # Load + filter
    chunks = []
    with jsonlines.open(CHUNK_JSONL) as r:
        for chunk in r:
            if chunk.get('source_type') == 'web_page':
                if not should_skip(chunk):
                    chunks.append(chunk)
    print(f'After filter: {len(chunks)} web chunks')
    # Batch embed + upsert
    for i in tqdm(range(0, len(chunks), BATCH_SIZE), desc='Web Pages'):
        batch = chunks[i : i + BATCH_SIZE]
        texts = [c['text'] for c in batch]
        try:
            embeddings = embed_documents(texts)
            col.upsert(
                ids=[c['chunk_id'] for c in batch],
                embeddings=embeddings,
                documents=texts,
                metadatas=[build_metadata(c, 'web_page') for c in batch]
            )
            time.sleep(0.05) # polite delay
        except Exception as e:
            print(f'Batch {i} failed: {e} - retrying one-by-one...')
            time.sleep(10)
            for chunk in batch:
                try:
                    emb = embed_documents([chunk['text']])
                    col.upsert(
                        ids=[chunk['chunk_id']],
                        embeddings=emb,
                        documents=[chunk['text']],
                        metadatas=[build_metadata(chunk, 'web_page')]
                    )
                    time.sleep(0.5)
                except Exception as e2:
                    print(f' Skip {chunk["chunk_id"]}: {e2}')
    print(f'Web DONE. Total: {col.count()}')
```

1.4 Ingest PDF Collection

PART 4 — PDF Collection

```
def ingest_pdfs():
    col = chroma.get_or_create_collection(
        name=COLLECTION_PDF,
        metadata={'hnsw:space': 'cosine'}
    )
    chunks = []
    with jsonlines.open(CHUNK_JSONL) as r:
        for chunk in r:
            if chunk.get('source_type') == 'pdf':
                if not should_skip(chunk):
                    chunks.append(chunk)
    print(f'PDF chunks after filter: {len(chunks)}')
    for i in tqdm(range(0, len(chunks), BATCH_SIZE), desc='PDFs'):
        batch = chunks[i : i + BATCH_SIZE]
        texts = [c['text'] for c in batch]
        try:
            embeddings = embed_documents(texts)
            col.upsert(
                ids=[c['chunk_id'] for c in batch],
                embeddings=embeddings,
                documents=texts,
                metadatas=[build_metadata(c, 'pdf') for c in batch]
            )
            time.sleep(0.05)
        except Exception as e:
            print(f'PDF batch {i} error: {e}')
            time.sleep(10)
    print(f'PDF DONE. Total: {col.count()}')
```

1.5 Ingest Metadata Collection

PART 5a — Metadata Collection

```
def ingest_metadata():
    col = chroma.get_or_create_collection(
        name=COLLECTION_META,
        metadata={'hnsw:space': 'cosine'}
    )
    records = []
    with jsonlines.open(CORPUS_JSONL) as r:
        for rec in r:
            if rec.get('title') and rec.get('url'):
                records.append(rec)
    for i in tqdm(range(0, len(records), BATCH_SIZE), desc='Metadata'):
        batch = records[i : i + BATCH_SIZE]
        texts = [f"{rec['title']} {rec.get('url','')}" for rec in batch]
        try:
            embeddings = embed_documents(texts)
            col.upsert(
                ids=[f"meta_{rec['url'][:80]}" for rec in batch],
                embeddings=embeddings,
                documents=texts,
                metadatas=[{
                    'url': str(rec.get('url',''))[:300],
                    'title': str(rec.get('title',''))[:200],
                    'level': int(rec.get('level', 0)),
                    'parent_url': str(rec.get('parent_url',''))[:300],
                } for rec in batch]
            )
            time.sleep(0.05)
        except Exception as e:
            print(f'Meta batch {i} error: {e}')
    print(f'Metadata DONE. Total: {col.count()}')
```

1.6 Build BM25 Index + Main

PART 5b — BM25 + Main Entry Point

[illegible]

STEP 2

Retrieval Pipeline

nodes.py + graph.py + api.py — Per-query · 7 LangGraph nodes

7 Nodes: query_analyzer → hybrid_retriever → [multi_hop_retriever] → reranker → generator → hallucination_guard → response_formatter

Every node uses gemini-2.0-flash for LLM calls. Query embedding uses gemini-embedding-2-preview with RETRIEVAL_QUERY task type.

2.1 nodes.py — Part A: Setup + Helpers

```
# nodes.py — ALL 7 LANGGRAPH NODES
import json, pickle, time
from google import genai
from google.genai import types
import chromadb
from sentence_transformers import CrossEncoder
from state import RAGState
from config import *
```

```
# █ Initialize once at module load █
gemini_client = genai.Client(api_key=GOOGLE_API_KEY)
chroma = chromadb.PersistentClient(path=CHROMA_PATH)
col_web = chroma.get_collection(COLLECTION_WEB)
col_pdf = chroma.get_collection(COLLECTION_PDF)
col_meta = chroma.get_collection(COLLECTION_META)
reranker = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')
with open(BM25_INDEX, 'rb') as f:
    _bm25_data = pickle.load(f)
bm25_engine = _bm25_data['bm25']
bm25_chunks = _bm25_data['chunks']
```

```
# █ Core Helpers ██████████
```

```
def embed_query(text: str) -> list[float]:  
    """Query embedding – MUST use RETRIEVAL_QUERY (different from docs)."""  
    resp = gemini_client.models.embed_content(  
        model=EMBED_MODEL,  
        contents=[text],  
        config=types.EmbedContentConfig(task_type='RETRIEVAL_QUERY')  
    )  
    return resp.embeddings[0].values  
  
def llm_json(system: str, user: str) -> dict:  
    """Call Gemini 2.0 Flash and parse JSON response."""  
    resp = gemini_client.models.generate_content(  
        model=LLM_MODEL,  
        contents=user,  
        config=types.GenerateContentConfig(  
            system_instruction=system,  
            response_mime_type='application/json',  
            temperature=0.1,  
            max_output_tokens=2048,  
        )  
    )  
    return json.loads(resp.text)
```

2.2 Node 1: query_analyzer

NODE 1 — query_analyzer

```
ANALYZER_SYSTEM = '''
```

You are a query analysis expert for the Netsol Technologies knowledge base.

Netsol makes enterprise software: LeasePak, Appex Now, NTFC, NFS Ascent

for asset finance, leasing, fleet management, and fintech.

Corpus: 13.7M words – 6,528 web pages + 602 PDFs.

Analyze the query and return ONLY valid JSON – no preamble:

```
{
  "intent": "product_info|financial|technical|competitive|general",
  "complexity": "simple|complex|multi_hop",
  "route": "web_only|pdf_only|hybrid|competitive_mode",
  "persona": "executive|developer|sales|general",
  "rewritten_queries":["semantic variant","keyword variant","broad variant"],
  "pdf_priority": true or false,
  "time_sensitive": true or false,
  "metadata_filters": {"source_type":"","min_date":""}
}
```

ROUTING RULES:

- financial query OR keywords (revenue,report,annual,growth) → pdf_priority:true, hybrid
- query has latest/recent/current → time_sensitive:true
- competitor name in query → competitive_mode route
- query needs 2+ separate document types → multi_hop
- Expand: NTFC→Netsol Trade Finance, NFS→Netsol Financial Suite, LeasePak→LeasePak asset finance software, Appex→Appex Now platform

```
'''

def query_analyzer(state: RAGState) -> RAGState:
    result = llm_json(ANALYZER_SYSTEM, state['query'])
    return {**state, **result}
```

2.3 Node 2: hybrid_retriever

NODE 2a — hybrid_retriever: Vector Search

[illegible]

Node 2 continued: BM25 + Merge

```
# BM25 Keyword Search
tokens = state['query'].lower().split()
bm25_scores= bm25_engine.get_scores(tokens)
top_idxs = sorted(range(len(bm25_scores)),
key=lambda i: bm25_scores[i], reverse=True)[:10]
for idx in top_idxs:
    chunk = bm25_chunks[idx]
    cid = chunk.get('chunk_id', str(idx))
    if cid not in seen_ids:
        seen_ids.add(cid)
        all_results.append({
            **chunk,
            'rrf_score': bm25_scores[idx] / max(bm25_scores) * 0.8,
        })
# Filter noise + sort
clean = [r for r in all_results
if len(r.get('text', '').split()) >= MIN_CHUNK_WORDS]
clean.sort(key=lambda x: x['rrf_score'], reverse=True)
return {**state, 'retrieved_chunks': clean[:TOP_K_RETRIEVE]}
```

2.4 Node 3: multi_hop_retriever (complex queries only)

NODE 3 — multi_hop_retriever

```
def multi_hop_retriever(state: RAGState) -> RAGState:
    # Only runs for complex multi-hop queries
    if state.get('complexity') != 'multi_hop':
        return state
    top5 = state['retrieved_chunks'][:5]
    context = ' '.join([c['text'][:300] for c in top5])
    existing= {c['chunk_id'] for c in state['retrieved_chunks']}
    # Extract entities from first retrieval
    sys = 'Extract key entities. Return ONLY a JSON array of 3 strings.'
    usr = f'Query: {state["query"]}\nContext: {context}'
    entities = llm_json(sys, usr)
    if not isinstance(entities, list):
        entities = list(entities.values())[:3]
    extra = []
    for entity in entities:
        emb = embed_query(str(entity))
        for col in [col_web, col_pdf]:
            res = col.query(query_embeddings=[emb], n_results=4,
                include=['documents', 'metadatas', 'distances'])
            for i, doc in enumerate(res['documents'][0]):
                meta = res['metadatas'][0][i]
                cid = meta.get('chunk_id', '')
                if cid and cid not in existing:
                    existing.add(cid)
                    extra.append({
                        'chunk_id': cid,
                        'text': doc,
                        'source_url': meta.get('source_url', ''),
                        'page_title': meta.get('page_title', ''),
                        'source_type': meta.get('source_type', ''),
                        'pdf_name': meta.get('pdf_name', ''),
                        'page_number': meta.get('page_number', 0),
                        'rrf_score': 1 - res['distances'][0][i],
                    })
    combined = state['retrieved_chunks'] + extra
    combined.sort(key=lambda x: x['rrf_score'], reverse=True)
    return {'**state', 'retrieved_chunks': combined[:TOP_K_RETRIEVE]}
```

2.5 Node 4: reranker

NODE 4 — reranker

```
def reranker_node(state: RAGState) -> RAGState:
    chunks = state['retrieved_chunks']
    query = state['query']
    # Cross-encoder scores each chunk against original query
    pairs = [(query, c['text'][:512]) for c in chunks]
    scores = reranker.predict(pairs)
    ranked = sorted(zip(scores, chunks), reverse=True)
    top = [c for _, c in ranked[:TOP_K_RERANK]]
    return {**state, 'reranked_chunks': top}
```

2.6 Node 5: generator

NODE 5a — GENERATOR_SYSTEM Prompt

```
GENERATOR_SYSTEM = '''
You are an expert analyst for Netsol Technologies.
Answer questions using ONLY the provided context chunks.
Netsol products: LeasePak, Appex Now, NTFC, NFS Ascent.
RULES:
1. Only use information from provided context
2. Never add information not in context
3. Cite every fact: [Source: URL] or [PDF: filename, p.XX]
4. If context is insufficient say: I found limited information...
PERSONA ADAPTATION:
- executive: revenue, market position, strategic value
- developer: APIs, integration, technical specs
- sales: features, benefits, case studies, ROI
- general: clear, accessible language
CONFIDENCE SCORE (0.0-1.0):
= (avg chunk similarity * 0.5)
+ (number of sources * 0.1, max 0.3)
+ (0.2 if PDF corroborates a web source)
Return ONLY valid JSON:
{
  "answer": "full answer with inline citations",
  "sources_used": ["url" or "pdf:filename:pagenum"],
  "confidence": 0.85,
  "confidence_label": "High or Medium or Low",
  "answer_type": "factual or analytical or comparative or insufficient"
}
'''
```

NODE 5 — generator

```
def generator_node(state: RAGState) -> RAGState:
    chunks = state['reranked_chunks']
    history = state.get('chat_history', [])[-3:] # last 3 turns only
    # Build context string with citations
    ctx = ''
    for i, c in enumerate(chunks):
        if c.get('source_type') == 'pdf':
            src = f"[PDF: {c['pdf_name']}, p.{c['page_number']}]"
        else:
            src = f"[Source: {c['source_url']}]'"
        ctx += f'\n--- Chunk {i+1} {src} ---\n{c["text"]}\n'
    hist_str = '\n'.join([f"{m['role']}: {m['content']}"
                           for m in history])
    user_msg = (
        f'PERSONA: {state.get("persona", "general")}\n'
        f'HISTORY:\n{hist_str}\n'
        f'QUESTION: {state["query"]}\n'
        f'CONTEXT:\n{ctx}'
    )
    result = llm_json(GENERATOR_SYSTEM, user_msg)
    return {**state,
            'draft_answer': result.get('answer', ''),
            'sources_used': result.get('sources_used', []),
            'confidence': float(result.get('confidence', 0.5)),
            'answer_type': result.get('answer_type', 'factual')}
```

2.7 Node 6: hallucination_guard

NODE 6 — hallucination_guard

```

GUARD_SYSTEM = '''
You are a fact-verification specialist.
Verify every factual claim in the answer is in the context chunks.
CHECK: numbers, dates, product names, client names, percentages.
IGNORE: transitional phrases, hedging, logical inferences.
Return ONLY valid JSON:
{
  "verdict": "PASS or FAIL or PARTIAL",
  "unsupported_claims": ["list of unverified claims"],
  "flagged_count": 0,
  "action": "USE_AS_IS or REMOVE_FLAGGED or REGENERATE or ADD_CAVEAT"
}
RULES:
- FAIL or flagged_count > 2 → action must be REGENERATE
- flagged_count == 1 → action must be REMOVE_FLAGGED
- PASS → action must be USE_AS_IS
'''

def hallucination_guard(state: RAGState) -> RAGState:
    retry = state.get('retry_count', 0)
    if retry >= MAX_RETRIES:
        # Max retries hit – add caveat and move on
        caveat = ('\n\nNote: Some details could not be fully '
        'verified against available sources.*')
        return {**state,
        'draft_answer': state['draft_answer'] + caveat,
        'hallucination_verdict': 'PARTIAL',
        'hallucination_action': 'ADD_CAVEAT'}
    ctx = '\n'.join([c['text'][:400] for c in state['reranked_chunks']])
    usr = f'ANSWER:\n{state["draft_answer"]}\n\nCONTEXT:\n{ctx}'
    result = llm_json(GUARD_SYSTEM, usr)
    new_retry = retry + (1 if result.get('action')=='REGENERATE' else 0)
    return {**state,
    'hallucination_verdict': result.get('verdict', 'PASS'),
    'hallucination_action': result.get('action', 'USE_AS_IS'),
    'retry_count': new_retry}

```

2.8 Node 7: response_formatter

NODE 7 — response_formatter

```
def response_formatter(state: RAGState) -> RAGState:
    conf = state.get('confidence', 0.5)
    label = ('High' if conf >= 0.75 else
            'Medium' if conf >= 0.50 else 'Low')
    final = {
        'answer': state['draft_answer'],
        'sources': state['sources_used'],
        'confidence': round(conf, 2),
        'confidence_label': label,
        'persona': state.get('persona', 'general'),
        'intent': state.get('intent', 'general'),
        'chunks_used': len(state.get('reranked_chunks', [])),
        'answer_type': state.get('answer_type', 'factual'),
        'verified': state.get('hallucination_verdict', 'UNKNOWN'),
    }
    return {**state, 'final_response': final}
```

2.9 graph.py — LangGraph Wiring

```
# graph.py
```

[illegible]

graph.py — Complete

```
def run_query(query: str, persona: str='general',
chat_history: list=[]) -> dict:
import time
t0 = time.time()
state = RAGState(
query=query, persona=persona, chat_history=chat_history,
intent='', complexity='', route='', rewritten_queries=[],
pdf_priority=False, time_sensitive=False, metadata_filters={},
retrieved_chunks=[], reranked_chunks=[],
draft_answer='', sources_used=[], confidence=0.0, answer_type='',
hallucination_verdict='', hallucination_action='',
retry_count=0, final_response={}, processing_time=0.0, error=None
)
out = rag_graph.invoke(state)
out['final_response']['processing_time'] = round(time.time()-t0, 2)
return out['final_response']
```

2.10 api.py — FastAPI Server

api.py — Complete

```
# api.py — FastAPI + SSE Streaming
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import StreamingResponse
from pydantic import BaseModel
from typing import List, Optional
import json, asyncio
from graph import run_query
from nodes import col_web, col_pdf
from config import LLM_MODEL

app = FastAPI(title='Netsol RAG API', version='2.0')
app.add_middleware(CORSMiddleware, allow_origins=['*'],
allow_methods=['*'], allow_headers=['*'])
class QueryRequest(BaseModel):
    query: str
    persona: Optional[str] = 'general'
    chat_history: Optional[List[dict]] = []
# POST /query — standard response
@app.post('/query')
async def query_endpoint(req: QueryRequest):
    try:
        return run_query(req.query, req.persona, req.chat_history)
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
# POST /query/stream — SSE token streaming for frontend
@app.post('/query/stream')
async def stream_endpoint(req: QueryRequest):
    async def generate():
        yield f'data: {json.dumps({"stage": "analyzing"})}\n\n'
        await asyncio.sleep(0.1)
        result = run_query(req.query, req.persona, req.chat_history)
        yield f'data: {json.dumps({"stage": "streaming"})}\n\n'
        for word in result.get('answer', '').split(' '):
            yield f'data: {json.dumps({"stage": "token", "token": word + " "})}\n\n'
            await asyncio.sleep(0.02)
        yield f'data: {json.dumps({"stage": "complete", "result": result})}\n\n'
    return StreamingResponse(generate(), media_type='text/event-stream')
# GET /health
@app.get('/health')
async def health():
    return {'status': 'ok', 'web_chunks': col_web.count(),
'pdf_chunks': col_pdf.count(), 'model': LLM_MODEL}
# Run: uvicorn api:app --reload --port 8000
```

Final Run Commands — In Order:

1. `python ingest.py` → Run ONCE. Builds ChromaDB + BM25.
2. `uvicorn api:app --reload --port 8000` → Start server.
3. `POST http://localhost:8000/query` → Standard query.
4. `POST http://localhost:8000/query/stream` → Streaming for frontend.
5. `GET http://localhost:8000/health` → Check collections loaded.